

Cẩm nang vận hành đội AI từ xa

Hướng dẫn chi tiết claude-telegram-multi-relay

hailoc12

06/2026

Mục lục

Lời giới thiệu

Bạn vừa cài Claude Code. Nó mạnh — đọc file, sửa code, chạy lệnh, deploy. Nhưng mọi thứ xảy ra trong một cửa sổ terminal, và bạn phải ngồi trước máy đó mới dùng được.

Cuốn sách nhỏ này giải một bài toán cụ thể: **làm sao để điều khiển nhiều Claude Code cùng lúc, từ điện thoại, bất kể bạn đang ở đâu** — và làm sao để chúng tự chạy theo lịch, tự phục hồi khi sự cố.

Giải pháp là `claude-telegram-multi-relay` — một lớp điều phối (orchestration) xây trên đỉnh ccc (Claude Code Companion của tác giả kidandcat). Nó biến mỗi Claude thành một “nhân viên số” có riêng một kênh chat trên Telegram, để bạn giao việc, nhận báo cáo, lên lịch ca trực — giống cách bạn quản lý một đội người thật.

Cuốn sách dành cho:

- **Người đã dùng Claude Code** và muốn mở rộng từ một terminal sang một đội.
- **Solopreneur / chủ doanh nghiệp nhỏ** đang xây dựng AI Native Company — nơi một người vận hành nhiều AI Worker.
- **Kỹ sư** muốn tự động hóa công việc lặp lại qua Telegram và lập lịch định kỳ.

Sau khi đọc xong, bạn sẽ tự thiết lập được một đội Claude nhiều session, mỗi session một topic Telegram, có lịch tự chạy và tự chữa.

Điều kiện nền tảng: Sách hướng dẫn trên macOS (phần lập lịch dùng `launchd`). Phần relay lỗi chạy được trên nhiều nền tảng, nhưng bản hoàn chỉnh cần macOS.

Mục lục

1. Telegram Relay là gì
2. Kiến trúc hệ thống
3. Chuẩn bị môi trường và tạo bot Telegram
4. Cài đặt và cấu hình
5. Quản lý phiên (session)
6. Đa luồng: multi-topic, multi-user, multi-group

7. Phối hợp đội (broadcast)
8. Lập lịch định kỳ với launchd
9. Giám sát, tự phục hồi và kiểm toán
10. Tích hợp với AI Native Company
 - Phụ lục A: Bảng quy tắc R-00 đến R-27
 - Phụ lục B: Khắc phục sự cố
 - Phụ lục C: Thuật ngữ

Chương 1: Telegram Relay là gì

Vấn đề của Claude Code

Claude Code là một giao diện dòng lệnh (CLI). Để dùng, bạn mở terminal, gõ lệnh, đọc kết quả ngay trên màn hình đó. Mô hình này tuyệt vời khi bạn đang ngồi trước máy, nhưng sụp đổ trong bốn tình huống thực tế:

- Bạn đang ở ngoài đường, chỉ có điện thoại. Bạn muốn nhờ Claude kiểm tra một log lỗi, nhưng máy để ở nhà hoặc trên server.
- Bạn có mười tác vụ cần chạy song song — nghiên cứu mười đối thủ, theo dõi mười luồng tích hợp liên tục (CI). Mỗi tác vụ cần một Claude riêng. Bạn không thể mở mười terminal rồi canh từng cái.
- Một tác vụ chạy ba mươi phút. Bạn không muốn ngồi chờ. Bạn muốn nó báo qua điện thoại khi xong, và bạn có thể trả lời tiếp ngay trên điện thoại.
- Bạn muốn Claude **tự chạy mỗi sáng tám giờ** để tổng hợp báo cáo, mà không cần bạn phải mở máy lên.

Tất cả đều quy về một nhu cầu: một kênh liên lạc giữa bạn và Claude, tách rời khỏi chiếc terminal.

Relay là trạm trung chuyển

Một relay (trạm trung chuyển) là phần mềm đứng giữa Claude Code và Telegram:

bạn (điện thoại) <--> Telegram <--> RELAY <--> Claude Code

Relay làm đúng hai việc:

1. **Hướng vào (inbound):** bạn gõ tin nhắn trong Telegram, relay đọc được, đẩy vào Claude đang chạy.
2. **Hướng ra (outbound):** Claude xuất kết quả, relay bắt lấy, gửi về Telegram.

Hai chiều độc lập, chạy thời gian thực. Kết quả: bạn và Claude trò chuyện qua Telegram y như chat với một đồng nghiệp — nhưng “đồng nghiệp” đó đọc và sửa file, chạy lệnh, triển khai dịch vụ ngay trên máy bạn.

💡 **MẸO:** Hãy nghĩ relay như một “trợ lý đứng cạnh máy bạn”. Bạn nhắn cho trợ lý qua Telegram, trợ lý đọc to cho Claude (đang chạy trong terminal), rồi chép câu trả lời của Claude gửi lại bạn. Relay tự động hóa vai trò đó.

Vì sao lại là Telegram

Có nhiều ứng dụng nhắn tin. Telegram được chọn vì bốn lý do thực dụng:

- **Có sẵn ở mọi nơi** — điện thoại, máy tính, trình duyệt.
- **Bot API mạnh và miễn phí** — tạo chủ đề (topic), gửi file, nút tương tác (inline button), phát trực tiếp (streaming).
- **Forum Topics** — tính năng biến một nhóm thành nhiều kênh (giống Slack). Mỗi Claude được phân một topic riêng, tách bạch hoàn toàn.
- **Chế độ HTML** — định dạng đẹp: khối mã, chữ đậm, gạch đầu dòng.

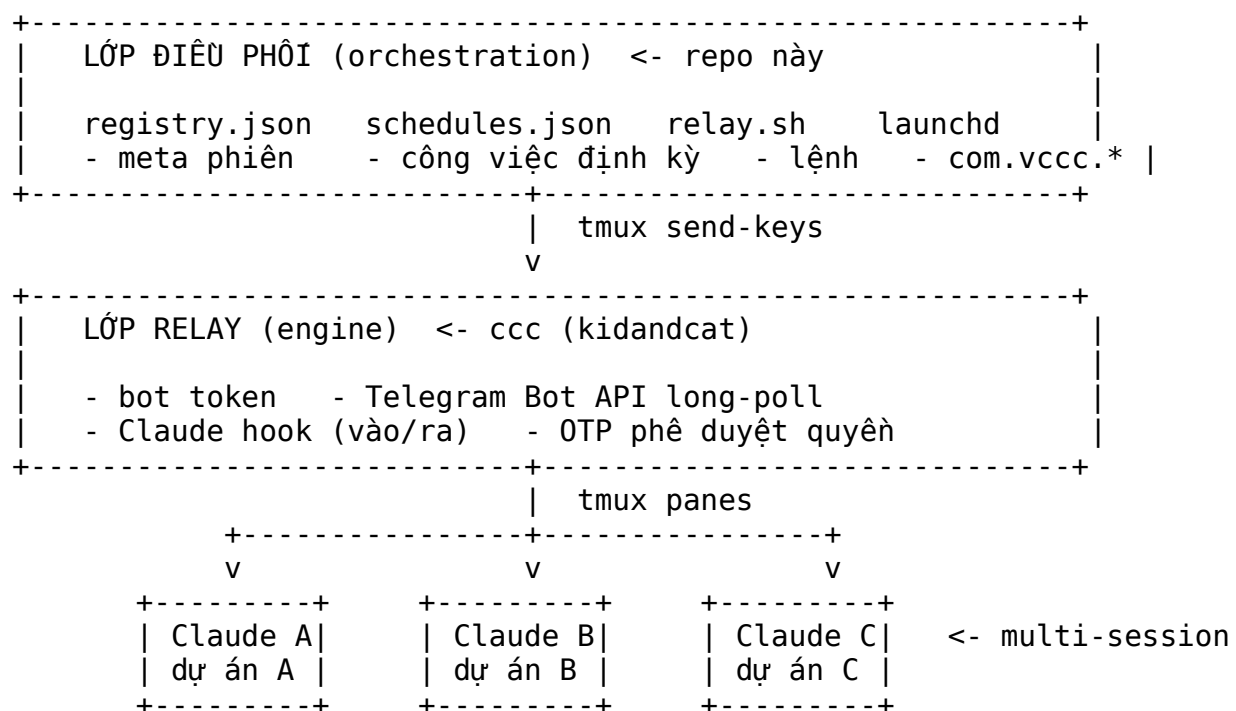
Một lợi thế tinh tế hơn: Telegram không giới hạn khổ như email hay tin nhắn điện thoại, nên phù hợp khi Claude gửi nhiều tin nhắn nhỏ liên tục.

🔑 **ĐIỂM CHÍNH:** Relay không thay thế Claude Code. Nó chỉ thêm một kênh điều khiển từ xa. Claude vẫn chạy trên máy bạn, với toàn bộ quyền truy cập file và hệ thống.

Chương 2: Kiến trúc hệ thống

Hai lớp tách biệt

Hiểu kiến trúc là chìa khóa để dùng và gỡ rối. Hệ thống gồm hai lớp xếp chồng:



Nguyên tắc quan trọng: repo này KHÔNG viết lại relay. Nó đọc và ghi cấu hình của ccc (nguồn dữ liệu chính cho kết nối), rồi thêm ba thứ ccc không có: registry (siêu dữ liệu điều phối), schedules (công việc định kỳ), scripts (vòng đời phiên).

ccc — phần nền móng

ccc (Claude Code Companion) là engine relay do tác giả kidandcat phát triển, mã nguồn mở tại github.com/kidandcat/ccc. Nó lo những việc nền tảng:

- Quản lý bot token và lắng nghe Telegram Bot API.
- Cài đặt hook vào Claude để bắt tin nhắn vào và ra.
- Phê duyệt quyền qua OTP (mã một lần) — khi Claude xin quyền chạy lệnh nhạy cảm, bạn duyệt ngay trên Telegram.
- Chuyển tiếp file và các lệnh `/new`, `/continue`, `/c`.

Phần này ổn định, đã qua thử nghiệm thực tế. Repo của chúng ta kế thừa nó, không tái chế tạo.

Lớp điều phối — phần repo này đóng góp

ccc tự thân xử lý tốt **một phiên ứng với một topic**. Khi bạn muốn nhiều phiên chạy song song, tự lên lịch, tự phục hồi — ccc chưa có cơ chế. Đó là khoảng trống repo này lấp:

Đóng góp	Mô tả
Điều phối multi-session	registry + vòng đời (tạo/gửi/list/trạng thái/xóa)
Multi-topic	tự tạo topic Telegram, map một-một với mỗi phiên
Multi-user / multi-group	một bot phục vụ nhiều nhóm, mỗi phiên nhớ <code>group_id</code>
Lập lịch bền vững	biểu thức cron -> launchd, sống qua khởi động lại và thức giấc
Quản lý đội	giám sát, broadcast, tự khởi động lại, kiểm toán

Bốn nguồn dữ liệu phải đồng bộ

Hệ thống có bốn nơi lưu trạng thái. Khi bạn thay đổi gì, phải cập nhật đủ, nếu không sẽ lệch (rule R-12):

Nguồn	Vai trò
<code>ccc config.json</code>	Kết nối: token, <code>group_id</code> , danh sách phiên (topic/path/window)
<code>registry.json</code>	Siêu dữ liệu điều phối: mục đích, thẻ, trạng thái, tự khởi động
<code>schedules.json</code>	Công việc định kỳ: biểu thức cron, prompt, lịch sử chạy

Nguồn	Vai trò
com.vccc.*.plist	Định nghĩa tác vụ launchd

Lệnh `relay_cron_list` tự phát hiện “mồ côi” — plist tồn tại mà không có entry trong registry, hoặc ngược lại — để bạn dọn dẹp.

⚠ **LƯU Ý:** Đừng bao giờ sửa tay bốn file này một cách tùy tiện. Luôn dùng các lệnh `relay_*`. Chúng đảm bảo đồng bộ. Sửa tay là nguyên nhân số một gây lệch trạng thái.

Chương 3: Chuẩn bị môi trường và tạo bot Telegram

Kiểm tra điều kiện tiên quyết

Trước khi cài, đảm bảo máy đã có đủ công cụ. Mở terminal và chạy từng lệnh kiểm tra:

```

claude --version      # Claude Code CLI
ccc --version          # relay engine (kidandcat/cc)
tmux -V              # qua'n lý cu'a sô' terminal
python3 --version     # ngôn ngữ kịch ba'n

```

Nếu thiếu cái nào:

```

npm i -g @anthropic-ai/claude-code  # Claude Code
brew install tmux python            # tmux + python3

```

`ccc` phải cài theo hướng dẫn của tác giả kidandcat tại github.com/kidandcat/cc. Đây là phụ thuộc bắt buộc — repo này điều phối `ccc`, không thay thế nó.

💡 **MẸO:** Chạy `ccc doctor` sau khi cài. Lệnh này kiểm tra toàn bộ phụ thuộc và cấu hình, báo rõ cái nào thiếu. Khắc phục hết trước khi đi tiếp.

Tạo bot Telegram

Bot là danh tính của hệ thống trên Telegram. Tạo một lần:

1. Mở Telegram, tìm **@BotFather**.
2. Gõ `/newbot`. Đặt tên (ví dụ “Đội AI của tôi”) và username phải kết thúc bằng `bot` (ví dụ `my_ai_team_bot`).
3. BotFather trả về một **token API** dạng `123456789:ABCdef...`. Đây là chìa khóa điều khiển bot — giữ kín, đừng đẩy lên Git.

`123456789:AAH-xyz...ABCdef` <- token, dánng thê' này

Tạo nhóm (group) có Topics

Tiếp theo, tạo không gian làm việc — một nhóm có bật tính năng Forum Topics:

1. Tạo nhóm mới trong Telegram, thêm bot vừa tạo vào làm **quản trị viên**.
2. Vào cài đặt nhóm, bật **Topics** (cần nhóm dạng supergroup).
3. Giờ nhóm trở thành forum — mỗi phiên Claude sẽ tự động là một topic.

Lấy **group_id** (số định danh nhóm) bằng cách thêm tạm bot @getidsbot hoặc @RawDataBot vào nhóm. Nó sẽ báo một số dạng -100xxxxxxxxxx. Ghi lại rồi xóa bot đó đi.

⚠ **LƯU Ý:** group_id luôn âm và bắt đầu bằng -100 — đó là supergroup. Đúng rồi, đừng tự ý bỏ dấu trừ.

Cấu hình token vào ccc

Token không nằm trong repo này — nó nằm trong cấu hình của ccc (nguồn dữ liệu chính). Cấu hình một lần:

```
ccc setup <DÁN_TOKEN_TỪ_BOTFATHER_VÀO_ĐÂY>
```

Lệnh ccc setup thực hiện ba việc: xác thực token, cài hook vào Claude, đăng ký dịch vụ launchd để lắng nghe Telegram. Sau đó cấu hình nhóm:

```
ccc setgroup      # nhập group_id để /new tạo topic hoạt động
ccc doctor        # kiểm tra lần cuối
```

Kiểm tra cấu hình đã nhận:

```
cat ~/.config/ccc/config.json | python3 -m json.tool
```

Bạn phải thấy bot_token, group_id, chat_id. Nếu đủ, phần nền móng xong.

Chương 4: Cài đặt và cấu hình

Tải repo về

```
git clone https://github.com/hailoc12/claude-telegram-multi-relay.git
cd claude-telegram-multi-relay
```

Cấu trúc thư mục:

```
claude-telegram-multi-relay/
├── SKILL.md                # Skill Claude Code (logic + quy tắc)
├── scripts/
│   ├── relay.sh            # thư viện lỗi: tạo/gửi/broadcast/cron
│   ├── run-scheduled.sh   # cầu nối launchd <-> Claude
│   └── cron_to_launchd.py  # chuyển biểu thức cron sang plist
├── config/                 # ví dụ cấu hình
├── templates/              # ví dụ launchd agent
└── docs/                   # tài liệu chuyên sâu
```

Cài đặt Skill vào Claude Code

Để Claude hiểu các lệnh điều phối, cài SKILL.md vào nơi Claude Code đọc skill:

```
SKILL_DIR=~/.claude/skills/claude-telegram-multi-relay
mkdir -p "$SKILL_DIR"
cp SKILL.md scripts -t "$SKILL_DIR"
```

Từ giờ, trong một phiên Claude Code, bạn có thể nói tự nhiên: “tạo ba phiên nghiên cứu đối thủ” hoặc “lên lịch báo cáo mỗi sáng tám giờ”.

Cài đặt cầu nối lập lịch

Phần lập lịch cần một thư mục dữ liệu và script cầu nối:

```
ORCH=~/.claude/ccc-orchestrator
mkdir -p "$ORCH/logs"
cp scripts/run-scheduled.sh "$ORCH/"
cp config/relay.example.env "$ORCH/relay.env"
```

Tùy chọn cấu hình (env)

File `relay.env` cho phép ghi đè đường dẫn nếu máy bạn khác mặc định. Hầu hết người dùng không cần đụng. Chỉ bỏ dấu thăng (#) và sửa khi:

- Binary ccc không nằm trong PATH (đặt `CCC_BIN`).
- Thư mục dự án gốc khác mặc định (đặt `PROJECTS_DIR`).

```
# Ví dụ ghi đè:
#CCC_BIN="/usr/local/bin/ccc"
#PROJECTS_DIR="$HOME/Projects"
```

Mặc định `PROJECTS_DIR` trỏ tới thư mục iCloud Documents/9. active (đồng bộ qua thiết bị). Nếu bạn dùng thư mục khác, đặt cho đúng.

Kiểm tra cài đặt

Chạy kiểm tra cuối:

```
source scripts/relay.sh
relay_list      # chưa có phiên nào cũng đúng – chỉ cần không báo lỗi
```

Nếu lệnh chạy mà không lỗi phụ thuộc, cài đặt thành công.

 **BÀI TẬP:** Tạo phiên đầu tiên ngay để kiểm tra toàn bộ đường truyền. Chạy `relay_new test-session "Kiểm tra kết nối"`, rồi vào nhóm Telegram xem bot có tự tạo topic và gửi thông báo sẵn sàng hay không. Nếu thấy — toàn bộ đường ống đã thông.

Chương 5: Quản lý phiên (session)

Phiên (session) là đơn vị cơ bản: một Claude Code chạy trong một cửa sổ tmux, gắn với một topic Telegram và một thư mục dự án. Phần này đi qua vòng đời phiên.

Tạo phiên mới

```
source scripts/relay.sh
relay_new ten-session "Mục đích của phiên"
```

Khi bạn chạy lệnh này, hệ thống thực hiện chín bước tự động:

1. Kiểm tra tên hợp lệ (chữ thường, gạch ngang, chưa tồn tại).
2. Tạo thư mục dự án, kèm `.claude/settings.json` cho phép bỏ qua prompt quyền.
3. Tạo topic Telegram qua API, nhận `topic_id`.
4. Mở cửa sổ tmux mới.
5. Khởi động Claude trong cửa sổ đó qua `ccc run`.
6. Đổi tên cửa sổ tmux về đúng tên phiên.
7. Ghi vào cấu hình `ccc` (topic, đường dẫn, cửa sổ).
8. Ghi vào registry (mục đích, trạng thái active).
9. Gửi thông báo sẵn sàng vào topic.

Bạn sẽ thấy trên Telegram:

✅ Session `ten-session` ready
Purpose: Mục đích của phiên
Dir: `/path/to/project`
Window: `@19`

Và trong nhóm xuất hiện một topic mới tên `ten-session`. Mọi tin nhắn bạn viết trong topic đó đi thẳng vào Claude.

💡 **MEO:** Quy tắc đặt tên: chữ thường, gạch ngang, ngắn gọn. Tên tốt: `comp-research-a`, `daily-report`, `ci-monitor`. Tên xấu: `My Session!`, chứa dấu tiếng Việt hoặc ký tự lạ.

Gửi lệnh tới phiên

Có hai cách. Cách tự nhiên nhất: gõ trực tiếp trong topic Telegram — Claude nhận ngay. Cách thứ hai, từ terminal:

```
relay_send ten-session "Phân tích điểm yếu đối thủ A, lưu ra report.md"
```

Hệ thống kiểm tra phiên còn sống, đẩy prompt vào cửa sổ tmux, cập nhật mốc thời gian lệnh cuối. Nếu cửa sổ đã chết, nó xử lý theo cài đặt tự khởi động lại.

Liệt kê toàn bộ phiên

```
relay_list
```

Kết quả dạng bảng, cho bạn bức tranh toàn đội:

📅 Sessions (4 registered)

<code>comp-research-a</code>	● active	<code>win:@19</code>	<code>topic:785</code>	Nghiên cứu đối thủ A
<code>comp-research-b</code>	● active	<code>win:@20</code>	<code>topic:786</code>	Nghiên cứu đối thủ B
<code>daily-report</code>	● idle	<code>win:@21</code>	<code>topic:787</code>	Báo cáo hàng ngày
<code>dead-session</code>	● dead	<code>win:@22</code>	<code>topic:788</code>	Phiên đã tắt

Trạng thái được suy ra từ việc cửa sổ tmux có còn tồn tại hay không: xanh khi đang chạy, vàng khi nhàn rỗi, đỏ khi đã tắt, xám khi chưa theo dõi.

Kiểm tra sức khỏe một phiên

```
relay_status ten-session
```

Lệnh kiểm tra ba lớp: cửa sổ tmux còn không, tiến trình Claude có đang chạy trong pane không, nội dung pane có dấu hiệu lỗi không. Trả về trạng thái kèm mã thoát, hữu ích khi viết script tự động.

Xóa phiên

```
relay_kill ten-session
```

Hệ thống gửi Ctrl-C rồi `/exit`, đợi, ép đóng cửa sổ nếu còn, dọn cấu hình ccc và đánh dấu registry là `terminated`. Khi dùng qua skill, bước này sẽ hỏi xác nhận bằng nút bấm trên Telegram (rule R-01) — tránh xóa nhầm.

Chuyển thư mục dự án

Đây là thao tác tinh tế: bạn muốn đổi thư mục làm việc của một phiên nhưng **giữ nguyên topic** — để lịch sử chat và danh tính phiên không đổi.

Đây là thao tác **tạo lại (recreate)**, không phải chỉ cập nhật cấu hình (rule R-23). Trình tự:

1. Đọc thông tin cũ (cửa sổ, topic, đường dẫn).
2. Đổi tên cửa sổ cũ thành `ten-session-old` (tránh nhầm khi tìm kiếm).
3. Tạo thư mục mới, kèm cấu hình bỏ quyền.
4. Mở cửa sổ tmux mới, chạy `ccc run`.
5. Cập nhật cấu hình ccc: đường dẫn + cửa sổ mới, **giữ nguyên topic_id**.
6. Cập nhật registry.
7. Đóng cửa sổ cũ, kiểm tra đã thực sự tắt.
8. Thông báo vào topic.

Sau bước này, bạn vẫn chat trong cùng một topic, nhưng Claude giờ làm việc trong thư mục mới.

⚠ **LƯU Ý:** Sai lầm phổ biến (và là lý do rule R-23 tồn tại): chỉ cập nhật cấu hình mà không tạo cửa sổ tmux mới, không chạy `ccc run`, không đóng phiên cũ. Kết quả: cấu hình nói thư mục mới nhưng Claude vẫn đang chạy ở thư mục cũ. Phải làm đủ bảy bước.

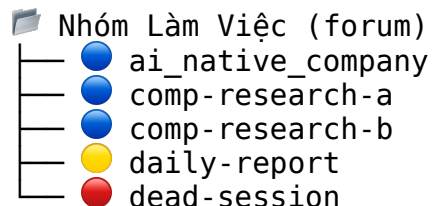
Chương 6: Đa luồng: multi-topic, multi-user, multi-group

Đây là phân tạo nên sự khác biệt cốt lõi của repo. Một relay đơn giản chỉ nối một Claude với một chat. Repo này thêm ba chiều “multi”.

Multi-session — nhiều Claude song song

Mỗi phiên là một Claude riêng: thư mục riêng, ngữ cảnh riêng, lịch sử riêng. Bạn có thể chạy mười phiên cùng lúc (thực tế giới hạn bởi RAM và CPU, thường mười đến hai mươi phiên trên một máy thường).

Khi bạn tạo nhiều phiên, mỗi cái tự động có một topic Telegram. Mở nhóm forum, bạn thấy:



Click vào topic nào là đang nói chuyện với phiên đó. Tách bạch, rõ ràng.

Multi-topic — mỗi phiên một kênh

Telegram Forum Topics biến một nhóm thành nhiều kênh (giống Slack channels). Khi tạo phiên, hệ thống gọi API `createForumTopic`, nhận `topic_id`, và map một-một phiên với topic.

Hướng ra: khi Claude trả lời, hook của ccc tra `topic_id` của phiên hiện tại, gửi vào đúng topic. Bạn không bao giờ bị lẫn phản hồi giữa các phiên.

Multi-user / multi-group — một bot, nhiều không gian

Đây là chiều mạnh nhất. **Một bot token có thể phục vụ nhiều nhóm khác nhau** — Telegram Bot API vốn hỗ trợ tự nhiên. Mỗi phiên lưu riêng `group_id`:

```
phiên "work-x"    -> group_id -1003460044206   (Nhóm Công việc)
phiên "family"    -> group_id -1003918647978   (Nhóm Gia đình)
```

Cơ chế routing rất đơn giản:

```
nhóm thực tế' = phiên.group_id  nếu có
               = config.group_id nếu phiên chưa có (fallback)
```

Ý nghĩa: bạn có thể có một nhóm cho công ty A, một nhóm cho khách hàng B, một nhóm cá nhân — tất cả dùng chung một bot, nhưng mỗi không gian hoàn toàn tách biệt. Đây chính là “multi-user”.

Thiết lập multi-group

1. Thêm bot vào nhiều nhóm (mỗi nhóm bật Topics, bot làm quản trị viên).
2. Trong mỗi nhóm, chạy `/new <tên>` — phiên tự gắn với nhóm đó.
3. Xong. Bot phục vụ tất cả các nhóm song song.

🔑 **ĐIỂM CHÍNH:** Khả năng tương thích ngược: phiên cũ không có `group_id` tự động dùng `group_id` mặc định ở mức cấu hình. Bạn không cần di chuyển dữ liệu khi nâng cấp lên multi-group.

Chương 7: Phối hợp đội (broadcast)

Khi bạn có nhiều phiên, sẽ có lúc cần gửi cùng một lệnh cho nhiều phiên cùng lúc. Đó là broadcast.

Cú pháp

```
relay_broadcast "prompt cần gửi" [mục tiêu]
```

Mục tiêu có ba dạng:

- `all` — tất cả phiên đang active.
- `group:tên` — các phiên thuộc một nhóm logic.
- `tag:tên` — các phiên mang một thẻ nào đó.

Ví dụ: đội nghiên cứu song song

Giả sử bạn muốn nghiên cứu năm đối thủ cùng lúc:

```
for c in a b c d e; do
  relay_new "comp-$c" "Nghiên cứu đối thủ $c"
done

relay_broadcast \
  "Nghiên cứu sâu: vị thế thị trường, giá, tính năng, điểm yếu. Lưu ra
  output/<tên>.md" \
  all
```

Năm Claude nhận cùng một lệnh, làm việc song song, mỗi cái lưu kết quả vào thư mục riêng. Bạn theo dõi tiến độ qua năm topic trên Telegram.

Nhóm và thẻ

Bạn có thể gán phiên vào nhóm logic hoặc gắn thẻ, rồi broadcast theo nhóm:

```
# (Qua skill hoặc chi'nh registry.json)
# nhóm "research-team" chứa comp-a, comp-b, comp-c
```

```
relay_broadcast "Tổng hợp phát hiện chính, mỗi phiên 3 điểm"
group:research-team
```

Cách này giúp tổ chức đội theo dự án hoặc theo vai trò.

Giới hạn và an toàn

Broadcast bị giới hạn mười phiên cùng lúc (rule R-07) để tránh quá tải hệ thống. Mỗi phiên được gửi độc lập, lỗi ở phiên này không ảnh hưởng phiên kia. Kết thúc, bạn nhận báo cáo dạng “đã gửi N, thất bại M”.

⚠ **LƯU Ý:** Broadcast gửi prompt bằng nhau cho mọi phiên. Nếu các phiên có ngữ cảnh rất khác nhau, cân nhắc gửi riêng từng cái hoặc dùng thẻ để nhóm các phiên tương đồng.

Chương 8: Lập lịch định kỳ với launchd

Đây là tính năng biến đội Claude từ “chạy khi được gọi” thành “tự chạy theo lịch”. Phần này giải thích vì sao dùng launchd chứ không phải cron, và cách vận hành.

Vì sao launchd, không phải cron

macOS có hai bộ lập lịch: cron (cũ) và launchd (native). Repo này mặc định launchd vì bốn lý do:

- **Apple đã ngừng khuyến khích cron** — launchd là bộ lập lịch gốc của macOS.
- **Thức giấc từ chế độ ngủ** — cron bỏ qua tác vụ khi Mac ngủ; launchd chạy khi máy thức dậy. Điều này quan trọng với máy để qua đêm.
- **Sống qua khởi động lại** — plist trong ~/Library/LaunchAgents tự nạp lại.
- **Một plist = một tác vụ** — dễ quản lý, sao lưu, chia sẻ, nhóm bằng tiền tố.

Hơn nữa, bản thân ccc đã chạy dưới dạng dịch vụ launchd, nên dùng cùng cơ chế cho nhất quán.

Cú pháp quen thuộc cron

Người dùng quen với biểu thức cron năm trường phím: phút giờ ngày tháng thứ. Bạn viết bằng cú pháp quen thuộc này, một script chuyển đổi (cron_to_launchd.py) tự dịch sang plist launchd nội bộ.

Một số mẫu phổ biến:

Biểu thức cron	Ý nghĩa	Chuyển sang launchd
7 9 * * 1-5	9h07 các ngày thường	StartCalendarInterval (5 ngày)
*/15 * * * *	mỗi 15 phút	StartInterval 900
13 * * * *	phút 13 mỗi giờ	StartCalendarInterval Minute=13
0 18 * * 5	18h00 thứ Sáu	StartCalendarInterval Hour=18,Weekday=5

Toàn vòng đời CRUD

Quản lý tác vụ định kỳ đầy đủ tạo-đọc-sửa-xóa:

Tạo

```
relay_cron_create daily-standup "7 9 * * 1-5" daily-report \  
  "Tổng hợp standup hôm nay với số liệu hôm qua"
```

```
# Liệt kê (kiểm tra ca' trạng thái nạp launchd)
relay_cron_list
```

```
# Tạm dừng (giữ plist trên đĩa)
relay_cron_toggle daily-standup false
```

```
# Bật lại
relay_cron_toggle daily-standup true
```

```
# Xem nhật ký (launchd tự ghi stdout/stderr)
relay_cron_history daily-standup
```

```
# Xóa hẳn
relay_cron_delete daily-standup
```

Quy ước đặt tên

Mọi tác vụ launchd dùng nhãn `com.vccc.<tên>` (rule R-05). Nhờ tiền tố thống nhất, bạn liệt kê toàn bộ đội bằng:

```
launchctl list | grep com.vccc
```

Tên tác vụ không chứa dấu chấm (rule R-17) để tương thích với định dạng nhãn.

Cầu nối `run-scheduled.sh`

Khi launchd kích hoạt, nó gọi script `run-scheduled.sh <tên-tác-vụ>`. Script này:

1. Đặt PATH đầy đủ (launchd có PATH tối thiểu, sẽ thiếu `tmux/python` nếu không đặt — rule R-19).
2. Kiểm tra phụ thuộc (`tmux`, `python3`, `ccc`) trước khi chạy — báo lỗi rõ thay vì sụp đổ mơ hồ (rule R-21).
3. Đọc định nghĩa tác vụ từ `schedules.json`.
4. Kiểm tra phiên mục tiêu còn sống; nếu chết và bật `auto-create`, **tự tạo lại phiên** rồi mới gửi prompt.
5. Đẩy prompt vào cửa sổ `tmux`.
6. Cập nhật mốc chạy cuối và trạng thái.
7. Gửi thông báo vào topic Telegram.

💡 **MẸO:** Mọi hàm trong script cầu nối phải được định nghĩa **trước** lần gọi đầu tiên (rule R-20). Bash không “nâng” hàm như nhiều ngôn ngữ — định nghĩa hàm ở cuối mà gọi ở giữa sẽ sụp đổ âm thầm. Đây là một trong những bug khó bắt nhất.

Chương 9: Giám sát, tự phục hồi và kiểm toán

Đội đồng thì cần giám sát. Phần này lo việc “ai đang sống, ai đã chết, ai cần cứu”.

Giám sát toàn đội

Chạy kiểm tra sức khỏe tất cả phiên:

```
for s in $(relay_list --names); do relay_status "$s"; done
```

Hoặc lên lịch tự giám sát mỗi ba mươi phút:

```
relay_cron_create fleet-monitor "*/30 * * * *" monitor \  
"Kiểm tra sức khỏe toàn bộ phiên"
```

launchd sẽ đánh thức Mac nếu đang ngủ để chạy việc này — cron không làm được.

Tự phục hồi (self-heal)

Mỗi phiên trong registry có cờ `auto_restart`. Khi phiên chết và cờ đang bật, hệ thống tự khởi động lại: tạo lại cửa sổ tmux, chạy `ccc run`, cập nhật cấu hình. Có giới hạn ba lần mỗi giờ (rule R-09) — nếu chết quá nhiều, hệ thống báo con người thay vì vòng lặp khởi động vô tận.

Cầu nối lập lịch cũng có self-heal riêng: nếu phiên mục tiêu đã chết và tác vụ bật `auto_create_session`, nó tự tạo phiên rồi mới gửi prompt. Tác vụ định kỳ không bao giờ “bom” vì phiên tình cờ tắt.

Kiểm toán (audit)

Sau mỗi lần tạo hoặc sửa tác vụ, nên chạy kiểm toán (rule R-22). Kiểm toán quét toàn đội và báo cáo:

- Script cầu nối có đặt PATH đầy đủ không (R-19).
- Hàm được định nghĩa trước lần gọi không (R-20).
- Có kiểm tra phụ thuộc đầu không (R-21).
- Mỗi tác vụ có plist đã nạp vào launchd không.
- Phiên mục tiêu còn sống (hoặc sẽ tự tạo).
- `last_status` không phải lỗi.
- Phát hiện “mồ côi”: plist mà không có entry registry và ngược lại.

Kết quả dạng số:  N đạt,  M cảnh báo,  K thất bại” kèm danh sách cần sửa.

Nhật ký

launchd tự ghi stdout vào `.log` và stderr vào `.err` (thông qua `StandardOutPath` trong plist — rule R-14). Bạn không cần tự tee. Khi tác vụ lỗi, xem:

```
relay_cron_history daily-standup 30
```

Hiển thị ba mươi dòng cuối của cả stdout và stderr.

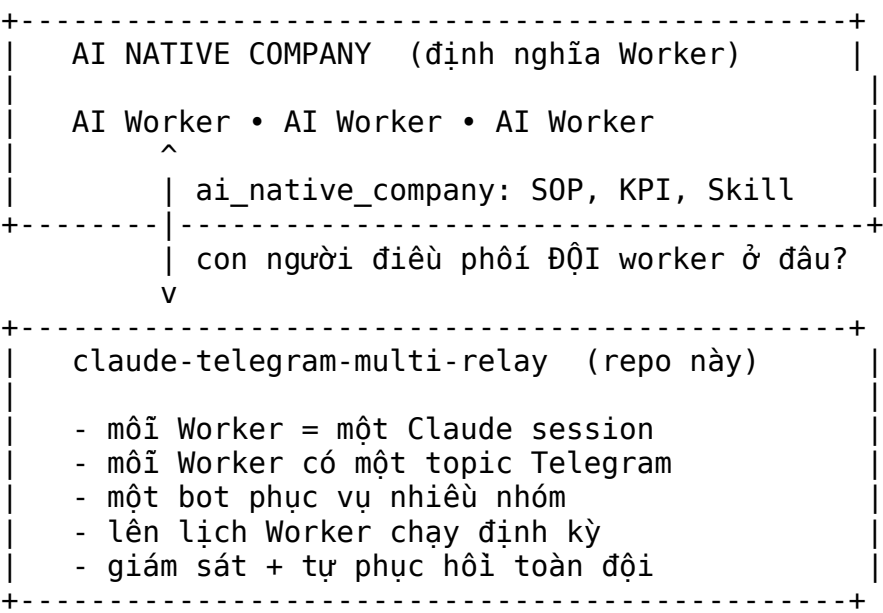
 **ĐIỂM CHÍNH:** Tự phục hồi + lập lịch bền vững + kiểm toán tạo nên một đội “tự quản”. Bạn có thể đi du lịch cả tuần, đội vẫn chạy đúng lịch, tự sửa khi sự cố, và bạn chỉ cần đọc báo cáo trên điện thoại.

Chương 10: Tích hợp với AI Native Company

Repo này không đứng đơn lẻ. Nó là **tầng giao tiếp và vận hành của con người** cho mô hình AI Native Company (github.com/hailoc12/ai_native_company).

Vị trí trong mô hình

AI Native Company định nghĩa **AI Worker** — nhân sự số chuyên biệt, mỗi worker có vai trò, quy trình (SOP), chỉ số chất lượng (SLI/KPI), kỹ năng và kiến thức. Một người có thể vận hành với năng lực tương đương mười lăm đến hai mươi người.



Nếu `ai_native_company` là **bộ não và sơ đồ tổ chức** (định nghĩa worker), thì repo này là **hệ thống liên lạc nội bộ cộng ca trực** — nơi con người điều phối đội mỗi ngày.

Ảnh xạ Worker thành phiên

Trong thực tế vận hành, mỗi AI Worker được triển khai thành một phiên Claude:

AI Worker (vai trò)	Phiên relay	Topic Telegram	Lịch
Worker Marketing	marketing-daily	topic riêng	standup 9h mỗi ngày
Worker Sales	sales-pipeline	topic riêng	tổng hợp 18h mỗi ngày
Worker Finance	finance-report	topic riêng	báo cáo 0h ngày 1 hàng tháng
Worker Research	research-fleet	nhiều topic	broadcast khi cần

Mỗi worker làm việc trong thư mục dự án riêng, có ngữ cảnh riêng, báo cáo vào topic riêng. Bạn — con người — mở Telegram và thấy toàn bộ đội đang làm gì.

Dòng chảy vận hành hàng ngày

Một ngày điển hình của người vận hành:

1. **Sáng:** mở Telegram, các worker đã tự chạy theo lịch, đọc báo cáo trong từng topic.
2. **Trong ngày:** giao việc cụ thể bằng cách nhắn trong topic tương ứng, nhận phản hồi theo thời gian thực.
3. **Chiều:** broadcast một chỉ thị chung cho cả đội (ví dụ “ưu tiên dự án X”).
4. **Tối:** kiểm toán đội, xử lý cảnh báo, lên lịch cho ngày mai.

Toàn bộ diễn ra trên điện thoại. Bạn không cần mở terminal, không cần ngồi trước máy. Đây chính là lời hứa của AI Native Company: một người, một đội, vận hành từ chiếc điện thoại trong tay.

Phụ lục A: Bảng quy tắc

Hai mươi bảy quy tắc (R-00 đến R-27) là bài học gỡ rối thực tế, đúc kết từ các lỗi đã gặp. Hiểu chúng giúp tránh lặp sai lầm.

Mã	Quy tắc
R-00	Đọc JSON bằng công cụ Read; python3 chỉ để ghi/chéo tham chiếu
R-01	Xác nhận trước khi xóa phiên (nút Telegram)
R-01b	Chuyển thư mục giữ nguyên topic_id
R-02	Tên phiên: chữ thường gạch ngang, duy nhất
R-03	Mọi phiên có một topic Telegram
R-04	launchd là bộ lập lịch chính; CronCreate chỉ cho tạm trong phiên
R-05	Mọi agent dùng nhãn <code>com.vccc.<tên></code>
R-06	plist nằm trong <code>~/Library/LaunchAgents/</code>
R-07	Broadcast tối đa mười phiên
R-08	Khoảng giám sát tối thiểu mười phút
R-09	Tự khởi động lại tối đa ba lần/giờ, rồi báo con người
R-10	Thoát dấu nháy đơn trong prompt send-keys
R-11	Kiểm tra phiên sống trước khi gửi
R-12	Bốn nguồn dữ liệu phải đồng bộ khi thay đổi
R-13	cron_list phát hiện mờ côi cả hai hướng
R-14	launchd tự ghi stdout/stderr (không cần tee)
R-15	cron_delete đỡ launchctl trước khi xóa plist
R-16	cron_update = đỡ → sửa → nạp lại (nguyên

Mã	Quy tắc
	từ)
R-17	Tên tác vụ không chứa dấu chấm
R-18	Luôn khởi động Claude qua <CCC_BIN> run
R-19	Script cầu nối đặt PATH ngay sau set -euo pipefail
R-20	Hàm định nghĩa trước lần gọi đầu tiên
R-21	Script cầu nối kiểm tra phụ thuộc đầu
R-22	Chạy kiểm toán sau mỗi lần tạo/sửa cron
R-23	Chuyển thư mục là tạo lại, không chỉ sửa cấu hình
R-24	Không bao giờ chuyển múi giờ: giờ cron = giờ plist, giờ địa phương
R-25	Thư mục mặc định \$PROJECTS_DIR/<tên>, trích dẫn nếu có khoảng trắng
R-26	Thư mục mới phải có .claude/settings.json bỏ quyền trước khi ccc run
R-27	Đổi tên cửa sổ tmux về tên phiên sau ccc run

 **ĐIỂM CHÍNH:** R-19, R-20, R-21 là bộ ba “PATH và thứ tự” — chúng bảo script cầu nối chạy đúng trong môi trường launchd tối giản. R-24 (không chuyển múi giờ) và R-26 (bỏ quyền) là hai bug khó tìm nhất nếu vi phạm.

Phụ lục B: Khắc phục sự cố

Bot không nhận tin nhắn vào. Chạy ccc doctor. Kiểm tra dịch vụ lắng nghe đang chạy: thử ccc listen thủ công. Xác nhận bot là quản trị viên trong nhóm và nhóm đã bật Topics.

Tác vụ cron không kích hoạt. Chạy relay_cron_list và nhìn hai cột “loaded” và “disk”. Nếu plist có trên đĩa nhưng chưa nạp: launchctl load ~/Library/LaunchAgents/com.vccc.<tên>.plist. Xem lỗi: tail ~/.claude/ccc-orchestrator/logs/<tên>.err.

Tác vụ chạy sai giờ (trễ/ba tiếng). Bạn đã chuyển múi giờ nhầm. launchd dùng **giờ địa phương**, không phải UTC (R-24). Nếu cron ghi 0 8 (tám giờ sáng), plist phải là Hour=8, không phải Hour=1.

Claude hỏi quyền liên tục qua Telegram. Thư mục dự án thiếu file bỏ quyền. Tạo mkdir -p <dir>/.claude && echo '{"permissionMode":"bypassPermissions"}' > <dir>/.claude/settings.json trước khi ccc run (R-26).

Script cầu nối báo “command not found” ở dòng lạ. Môi trường launchd có PATH tối giản. Đảm bảo script đặt `export PATH=...` ngay sau `set -euo pipefail` (R-19) và có khối kiểm tra phụ thuộc đầu (R-21).

Hàm không chạy / sụp đổ âm thầm. Hàm được gọi trước khi định nghĩa (R-20). Di chuyển mọi định nghĩa hàm lên trên mọi điểm gọi trong script cầu nối.

Cửa sổ tmux bị đổi tên sau khi tạo. `ccc` run có thể tự đổi tên cửa sổ. Force đổi lại: `tmux rename-window -t <id> <tên>` (R-27).

Lỗi “session already exists” nhưng không thấy phiên. Có entry mờ côi trong cấu hình `ccc`. Dọn bằng tay: xóa entry đó trong `~/ .config/ccc/config.json`, rồi chạy `relay_list` xác nhận.

Phụ lục C: Thuật ngữ

Relay: phần mềm đứng giữa Claude Code và Telegram, chuyển tin nhắn hai chiều.

Session (phiên): một Claude Code chạy trong một cửa sổ tmux, gắn với một topic và một thư mục dự án.

Topic: một kênh trong nhóm Telegram forum; mỗi phiên map một-một với một topic.

group_id: số định danh một nhóm Telegram (dạng -100xxxxxxxxxxx cho supergroup).

BotFather: bot chính thức của Telegram để tạo và quản lý bot khác.

launchd: bộ lập lịch native của macOS, thay thế cron. Sống qua khởi động lại và thức giấc từ chế độ ngủ.

plist (danh sách thuộc tính): file XML định nghĩa một tác vụ launchd.

cron: biểu thức năm trường (phút giờ ngày tháng thứ) mô tả lịch chạy.

registry.json: file lưu siêu dữ liệu điều phối (mục đích, thẻ, trạng thái) cho mỗi phiên.

schedules.json: file lưu các tác vụ định kỳ (cron, prompt, lịch sử).

broadcast: gửi cùng một lệnh cho nhiều phiên cùng lúc.

CCC (Claude Code Companion): engine relay của tác giả kidandcat, phần nền móng mà repo này xây trên.

AI Worker: nhân sự số chuyên biệt trong mô hình AI Native Company.

Cuốn sách này thuộc hệ sinh thái [AI Native Company](#). Repo và mã nguồn: [claude-telegram-multi-relay](#). Engine nền móng: [ccc \(kidandcat\)](#).